아키텍처 문서

Morning Momentum Scanner — 시스템 아키텍처

1. 전체 구성도

```
┌──────────────────────────────────────────────────────┐
│                    Client (Browser)                  │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │           Next.js App (React 19)             │    │
│  │                                              │    │
│  │  page.tsx (메인 UI)                          │    │
│  │    ├── MarketSessionBadge (세션 감지)        │    │
│  │    ├── RefreshCountdown (자동 갱신)          │    │
│  │    └── TickerCard[] (종목 카드 그리드)       │    │
│  │          ├── ScoreCriteriaBar (점수 바)      │    │
│  │          └── EntryStrategyBox (진입 전략)    │    │
│  └──────────────────────┬───────────────────────┘    │
│                         │ fetch /api/stock/[ticker]  │
└─────────────────────────┼────────────────────────────┘
                          │
┌─────────────────────────┼────────────────────────────┐
│             Next.js API Route (Server)               │
│                         │                            │
│  ┌──────────────────────▼───────────────────────┐    │
│  │  /api/stock/[ticker]/route.ts                 │    │
│  │    ├── Yahoo Finance API 호출                 │    │
│  │    │     (v8/finance/chart)                   │    │
│  │    ├── 성공 → extractTickerData()             │    │
│  │    └── 실패 → generateMockTicker()            │    │
│  └──────────────────────────────────────────────┘    │
│                                                      │
└──────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────┐
│             Yahoo Finance API (External)             │
│  https://query1.finance.yahoo.com/v8/finance/chart   │
└──────────────────────────────────────────────────────┘
```

2. 계층 구조

2.1 프레젠테이션 계층 (Presentation Layer)

클라이언트 사이드 React 컴포넌트로 구성됩니다.

컴포넌트	역할
`page.tsx`	메인 페이지 — 상태 관리, 데이터 페칭 조율

`TickerCard`	개별 종목 데이터 및 스코어 표시
`ScoreCriteriaBar`	7가지 기준별 점수 시각화
`EntryStrategyBox`	진입 전략 표시
`MarketSessionBadge`	현재 마켓 세션 상태 표시
`RefreshCountdown`	60초 자동 갱신 카운트다운

2.2 비즈니스 로직 계층 (Business Logic Layer)

`src/lib/` 디렉토리에 위치하며, 순수 함수로 구성됩니다.

모듈	역할
`scoring.ts`	7가지 기준에 따른 모멘텀 점수 계산
`marketSession.ts`	ET 기준 마켓 세션 감지
`mockData.ts`	API 실패 시 모의 데이터 생성
`constants.ts`	매직 넘버/스트링 상수화
`types.ts`	TypeScript 타입 정의

2.3 데이터 계층 (Data Layer)

계층	구성
API Route	`src/app/api/stock/[ticker]/route.ts` — Yahoo Finance 프록시
외부 API	Yahoo Finance v8 Chart API
폴백	`generateMockTicker()` — API 실패 시 자동 전환

3. 데이터 흐름

```
1. 사용자가 종목 심볼 입력 후 Scan 클릭
   └> page.tsx: handleScan()

2. 각 심볼에 대해 병렬 API 요청 (Promise.allSettled)
   └> fetch('/api/stock/TSLA')
   └> fetch('/api/stock/NVDA')
   └> ...

3. API Route에서 Yahoo Finance 호출
   ├─ 성공 → extractTickerData()로 파싱 → { isLive: true }
   └─ 실패 → generateMockTicker() → { isLive: false }

4. 클라이언트에서 응답 수신
   └> calculateScore(tickerData)로 7가지 기준 점수 계산
   └> generateEntryStrategy()로 진입 전략 생성
```

```
5. TickerCard 렌더링
   ├─ ScoreCriteriaBar × 7
   ├─ EntryStrategyBox
   └─ 시그널 배지 (HIGH / MODERATE / LOW)

6. 60초 후 자동 갱신 (RefreshCountdown)
   └─> 마지막 스캔한 심볼로 재요청
```

4. 스코어링 알고리즘

```
총점 = C1 + C2 + C3 + C4 + C5 + C6 + C7
       (각 0~2점, 총 0~14점)

시그널 판정:
  총점 >= 10  →  HIGH (강한 모멘텀)
  총점 >= 6   →  MODERATE (보통 모멘텀)
  총점 < 6    →  LOW (약한 모멘텀)
```

5. 에러 처리 전략

상황	처리 방식
Yahoo API 응답 실패	모의 데이터 폴백 + `isLive: false`
개별 종목 페칭 실패	`Promise.allSettled`로 나머지 종목은 정상 처리
데이터 파싱 오류	모의 데이터 폴백 + 에러 로그 출력
네트워크 오류	콘솔 에러 로그 + UI에서 빈 상태 유지

6. 성능 최적화

- **병렬 API 호출**: `Promise.allSettled` 로 모든 종목을 동시에 요청
- **서버사이드 프록시**: CORS 우회 및 클라이언트 부하 감소
- **CSS 인라인 테마**: Tailwind CSS v4의 `@theme inline` 으로 빌드 최적화
- **캐시 무효화**: `cache: "no-store"` 로 항상 최신 데이터 요청
- **정적/동적 분리**: 메인 페이지는 정적 빌드, API 라우트만 동적 처리

7. 보안 고려사항

- Yahoo Finance API 키가 노출되지 않도록 서버사이드에서만 호출
- User-Agent 헤더를 포함하여 API 차단 최소화
- 클라이언트에 민감한 정보가 전달되지 않음
- XSS 방지: React의 기본 이스케이핑 활용